



색칠 문제

문제

n 개의 영역으로 나누어진 도형을 색칠하는 경우의 수를 구한다.

인접한 영역은 서로 다른 색으로 칠한다.

x 개의 색을 사용하고, 모든 색을 사용할 필요는 없다.

문제 분석

이 문제는 '그래프' 자료구조를 이용하여 표현할 수 있다. 각 영역에 겹치지 않는 번호를 부여하고, 각 번호로 그래프의 노드를 구성한다. 서로 인접하는 영역이 존재한다면 인접하는 각 영역의 번호에 해당하는 두 노드를 간선으로 연결한다.

이렇게 하면 복잡했던 문제를 n 개의 색으로 어떠한 무방향 그래프에서 간선으로 연결된 두 노드의 색이 다르도록 모든 노드를 색칠하는 문제로 단순화할 수 있다.

이를 여러 경우로 나누어 생각해보았다.

무방향 그래프에서 노드들이 연결된 구조는 크게 4가지로 나뉠 수 있다.

1. 고립형
2. 직선형
3. 순환형
4. 혼합형

고립형의 경우 어느 노드와도 간선으로 연결되지 않은 것으로, 경우의 수는

$$x^n$$

이 된다.

직선형의 경우 양 끝 2개의 노드를 제외한 모든 노드가 2개의 간선으로 연결된 것으로, 경우의 수는

$$x \times (x - 1)^{n-1}$$

이 된다.

순환형의 경우 모든 노드가 2개의 간선으로 연결되어 있으며 순환 그래프의 형태를 띠게 된다. 이때 경우의 수는

$$(x - 1)^n + (-1)^n(x - 1)$$

이 된다.

혼합형의 경우 1, 2, 3번 형태가 모두 등장할 수 있는 형태이다.

따라서 혼합형을 1, 2, 3번 형태로 나누어서 계산하는 것이 최종 목표이다.

삭제-축소 이론

무방향 그래프의 색칠 문제에서 간선으로 연결된 두 노드를 선택한 뒤, 두 노드 사이의 간선을 제거했을 때의 경우의 수를 A, 두 노드를 병합하여 하나의 노드로 만들었을 때의 경우의 수를 B라 할 때, 원본 그래프를 색칠하는 경우의 수는 A-B 가 된다.

이때 두 노드를 병합한다 함은 두 노드 x, y와 연결된 모든 다른 노드에 대해 x, y와의 간선을 제거하고, 새로운 노드 z와 간선을 연결하는 것을 말한다.

이 삭제-축소 연산을 반복하게 되면 혼합형이었던 그래프가 고립형, 직선형, 순환형 중 하나로 변환된다.

따라서 재귀적인 방법을 이용해 경우의 수를 계산할 수 있게 된다.

소스코드

```
import copy

def Circle(n, x):
    if n % 2 == 0:
        return (x-1)**n + (x-1)
```

```

else:
    return (x-1)**n - (x-1)

def Linear(n, x):
    return x * ((x-1)**(n-1))

def Isolated(n, x):
    return x**n

def Merge(arr1, arr2):
    return list(set(arr1 + arr2))

def isCircle(m_graph, v_remove):
    degree_count = {}
    visited = set()

    for i in range(len(m_graph)):
        if i in v_remove:
            continue
        degree = 0
        for neighbor in m_graph[i]:
            if neighbor not in v_remove:
                degree += 1
        if degree != 2:
            return False
        degree_count[i] = degree

    if not degree_count:
        return False

    start_node = next(iter(degree_count))

    def dfs(node, parent):
        visited.add(node)
        for neighbor in m_graph[node]:
            if neighbor in v_remove or neighbor == parent:
                continue
            if neighbor in visited:

```

```

        continue
    dfs(neighbor, node)

dfs(start_node, -1)

return len(visited) == len(degree_count)

def isLinear(m_graph, v_remove):
    degree_count = {}
    visited = set()

    for i in range(len(m_graph)):
        if i in v_remove:
            continue
        degree = 0
        for neighbor in m_graph[i]:
            if neighbor not in v_remove:
                degree += 1
        degree_count[i] = degree

    ends = [node for node, deg in degree_count.items() if deg == 1]
    if len(ends) != 2:
        return False

    for node, deg in degree_count.items():
        if deg not in [1, 2]:
            return False

    def dfs(node, parent):
        visited.add(node)
        for neighbor in m_graph[node]:
            if neighbor in v_remove or neighbor == parent:
                continue
            if neighbor in visited:
                continue
            dfs(neighbor, node)

    dfs(ends[0], -1)

```

```

return len(visited) == len(degree_count)

def getIsolatedNodes(m_graph, v_remove):
    isolated_nodes = []
    for i in range(len(m_graph)):
        if i in v_remove:
            continue
        if all(neighbor in v_remove for neighbor in m_graph[i]):
            isolated_nodes.append(i)
    return isolated_nodes

def get_unique_edges(graph):
    edges = set()
    for node, neighbors in enumerate(graph):
        for neighbor in neighbors:
            if node < neighbor:
                edges.add((node, neighbor))
    return sorted(edges)

def DC_Mergefit(colors, m_graph, m_edge=[], v_remove=[], depth=1):
    if len(m_edge) == 0:
        m_edge = get_unique_edges(m_graph)
    isolated_nodes = getIsolatedNodes(m_graph, v_remove)
    if len(isolated_nodes) > 0:
        iso_val = Isolated(len(isolated_nodes), colors)
        activated_node_count = len(m_graph) - len(v_remove)

        if activated_node_count == len(isolated_nodes):
            return iso_val

    iv_remove = copy.deepcopy(v_remove)
    for i in range(len(isolated_nodes)):
        iv_remove.append(isolated_nodes[i])

    other_val = DC_Mergefit(colors, m_graph, m_edge, iv_remove, depth+1)
    return iso_val * other_val

```

```

if isCircle(m_graph, v_remove):
    return Circle(len(m_graph)-len(v_remove), colors)

if isLinear(m_graph, v_remove):
    return Linear(len(m_graph)-len(v_remove), colors)

x, y = m_edge[0]

l_graph = copy.deepcopy(m_graph)
l_graph[x].remove(y)
l_graph[y].remove(x)
l_edge = copy.deepcopy(m_edge)
del l_edge[0]

s_graph = copy.deepcopy(l_graph)
s_edge = copy.deepcopy(l_edge)
sm_cnt = Merge(s_graph[x], s_graph[y])
for i in range(len(s_graph[x])):
    s_graph[s_graph[x][i]].remove(x)
    p, q = min(x, s_graph[x][i]), max(x, s_graph[x][i])
    s_edge.remove((p, q))
s_graph[x] = []
for i in range(len(s_graph[y])):
    s_graph[s_graph[y][i]].remove(y)
    p, q = min(y, s_graph[y][i]), max(y, s_graph[y][i])
    s_edge.remove((p, q))
s_graph[y] = []

s_graph[y] = sm_cnt
for i in range(len(sm_cnt)):
    s_graph[sm_cnt[i]].append(y)
    p, q = min(y, sm_cnt[i]), max(y, sm_cnt[i])
    s_edge.append((p, q))
sv_remove = copy.deepcopy(v_remove)
sv_remove.append(x)

l_val = DC_Mergefit(colors, l_graph, l_edge, v_remove, depth+1)

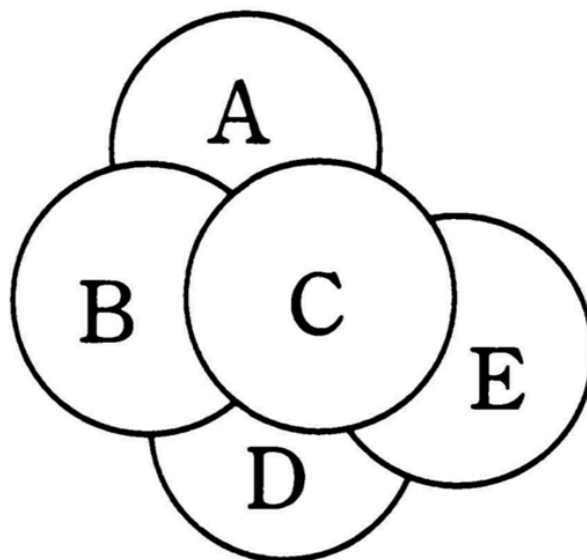
```

```
s_val = DC_Mergefit(colors, s_graph, s_edge, sv_remove, depth+1)
return l_val - s_val
```

<https://gist.github.com/airhood/60784b3d002de424db1f51f180d94569>

예제

예제 1



5개의 색 사용

정답

540

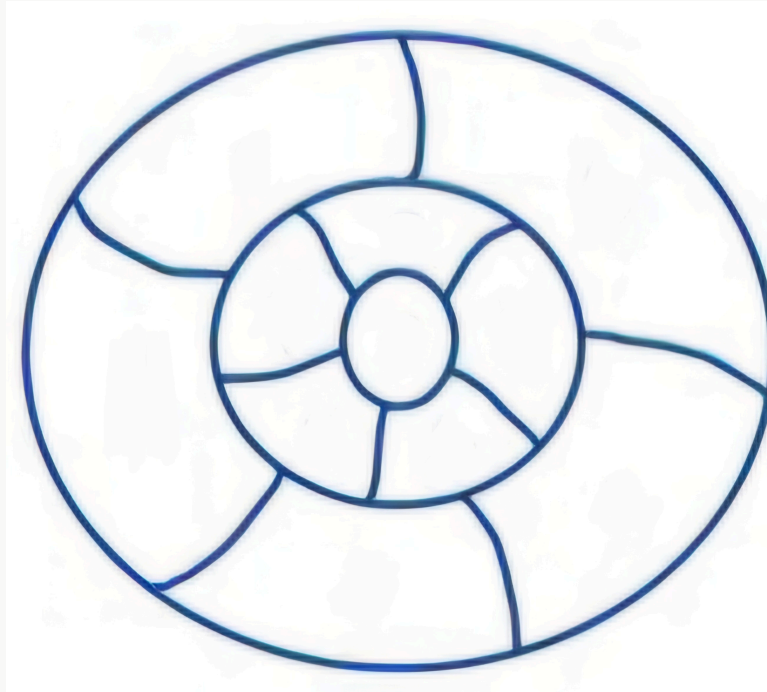
코드 구현

```
graph = [  
    [1, 2],  
    [0, 2, 3],  
    [0, 1, 3, 4],  
    [1, 2, 4],  
    [2, 3]  
]  
  
colors = 5  
  
result = DC_Mergefit(colors, graph)  
print(result)
```

실행 결과

540

예제 2



10개의 색 사용
(단, 배경은 색칠하지 않는다)

정답
(수학적 계산 아직 못함)

코드 구현

```
graph = [  
    [1, 2, 3, 4, 5],  
    [0, 2, 5, 6, 10],  
    [0, 1, 3, 6, 7],  
    [0, 2, 4, 7, 8],  
    [0, 3, 5, 8, 9],  
    [0, 1, 4, 9, 10],  
    [1, 2, 7, 10],  
    [2, 3, 6, 8],  
    [3, 4, 7, 9],  
    [4, 5, 8, 10],  
    [1, 5, 6, 9]  
]  
  
colors = 10  
  
result = DC_Mergefit(colors, graph)  
print(result)
```

실행 결과

5952693600

한계점

이 알고리즘은 경우의 수를 계산하는 연산 자체는 수학적인 방법을 사용하기에 색의 개수는 연산 시간에 영향을 주지 않는다. 하지만 그래프를 삭제-축소 하는 과정에서 재귀적인 방법을 사용하기 때문에 노드의 개수에 매우 큰 영향을 받는다. 또한 해당 그래프가 고립형, 직선형, 순환형 중 어느 형태에 속하는지 판단하기 위해 그래프 탐색이 진행되기 때문에 연산 시간을 더욱 증가시킨다.

하지만 실제로 연산 과정을 출력해보면 Y자나 T자 등 단순한 모양이 깊은 재귀 단계에서 많은 부분을 차지하고 있으며, 이를 삭제-축소 하는데에 많은 연산이 소요된다는 것을 알게 되었다. 따라서 동적계획법(Dynamic Programming)을 이용하여 반복되는 연산을 줄여보려 하였다.

하지만 이를 위해서는 두 그래프가 서로 동형인지 확인해야 하는데, 이를 해결하기 위한 빠른 알고리즘은 개발되지 않은 상태이다.

추가 탐구 방향

DP 적용을 시도해보기 위해 그래프 동형 문제를 해결할 수 있는 빠른 알고리즘을 구현하는 것에 대해 추가로 탐구해볼 예정이다.

또한 인접할 수 있는 색을 인접행렬의 형태로 제공했을 때, 경우의 수를 구하는 알고리즘에 대해서도 추가로 탐구해볼 것이다.

삭제-축소 이론을 이용한 재귀적인 방식이 아닌, 다른 방식에 대해서도 알아볼 것이다.